

ANALYSIS AND DESIGN OF ALGORITHMS

Instructor: Dr. Mert Büyükdede



Minimum Cost Spanning Trees (MCST)

Definition: A spanning tree of a connected graph is its connected acyclic subgraph (i.e., a tree) that contains all the vertices of the graph.

A minimum spanning tree of a weighted connected graph is its spanning tree of the smallest weight, where the weight of a tree is defined as the sum of the weights on all its edges.

The minimum spanning tree problem is the problem of finding a minimum spanning tree for a given weighted connected graph.



Main Definitions

Graph

A graph $G=(V,E)$ consists of a set of vertices V (or nodes) and a set of edges E . Each edge $e \in E$ connects two vertices and may have an associated nonnegative weight $w(e)$.

Connected Graph

A graph is called connected if, for every pair of vertices $u,v \in V$, there exists at least one path in G that leads from u to v .

Tree

A tree is a connected acyclic graph. Equivalently, a tree with $|V|$ vertices has exactly $|V|-1$ edges and contains no cycles.

Spanning Tree

Given a connected graph G , a spanning tree is a subgraph $T \subseteq G$ that includes all vertices of G , is connected, and has no cycles. Therefore T also has $|V|-1$ edges.

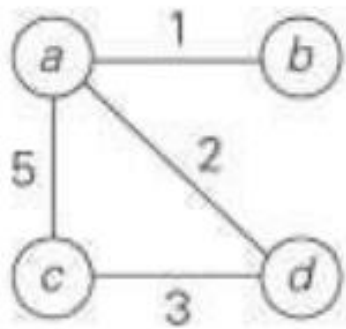
Minimum Spanning Tree (MST)

Among all spanning trees of a weighted, connected graph G , a minimum spanning tree is one whose total edge weight sum,

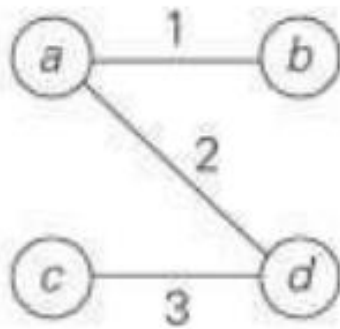
$$\sum_{e \in T} w(e),$$

is minimum. The goal is to find such a tree T .

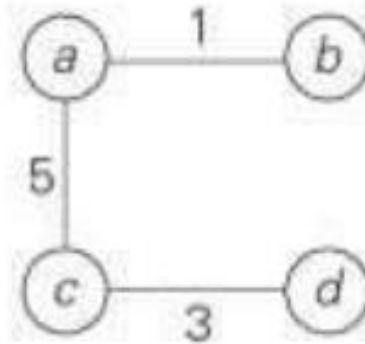
Examples of MST



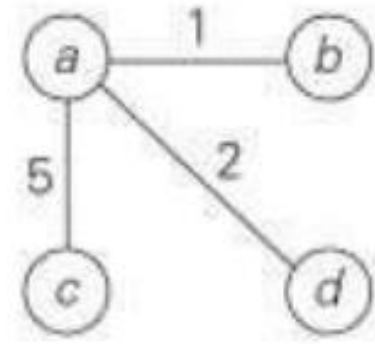
graph



$w(T_1) = 6$



$w(T_2) = 9$



$w(T_3) = 8$

Types of MST Algorithms

Below are the three most common greedy algorithms for finding a Minimum Spanning Tree:

Algorithm	Key Idea	Time Complexity
Kruskal	Sort all edges in increasing weight order, then add each edge if it does not form a cycle.	$O(E \log E)$ (using Union-Find)
Prim	Grow the tree one vertex at a time: at each step, add the smallest-weight edge connecting the tree to a new vertex.	$O(E \log V)$ (using a min-heap)
Borůvka	In parallel, for each component select its cheapest outgoing edge, merge components, and repeat until one tree remains.	$O(E \log V)$

Kruskal is often preferred for sparse graphs ($E \ll V^2$)

Prim is efficient on dense graphs ($E \approx V^2$)

Borůvka excels in large-scale or parallel implementations.

General Algorithm Framework and Common Steps

All MST algorithms share the same greedy principle:

At each step, choose the cheapest edge that can be added without violating the spanning-tree conditions (connectedness and acyclicity).

Common Concepts

•Cut Property

For any partition (“cut”) of the vertices into two sets S and $V \setminus S$, the minimum-weight edge crossing that cut belongs to some MST.

•Cycle Property

In any cycle, the heaviest edge cannot belong to any MST.

•Union-Find (Disjoint Set Union)

In Kruskal’s algorithm, supports

- **find(u)**: identify which component contains u ,
- **union(u,v)**: merge the components of u and v , both in nearly $O(\alpha(n))$ amortized time.

•Min-Heap / Priority Queue

In Prim’s algorithm, lets you quickly find and update the smallest-weight edge connecting the current tree to outside vertices.



Prim's Algorithm:

Goal: Build a spanning tree by “growing” outward from a starting vertex, always adding the cheapest edge that connects the current tree to a new vertex.

Key Idea: Maintain a set T of vertices that are already in the MST.

At each step, among all edges with one endpoint in T and the other outside T , choose the edge of minimum weight—and add its outside endpoint to T . Repeat until T contains every vertex.

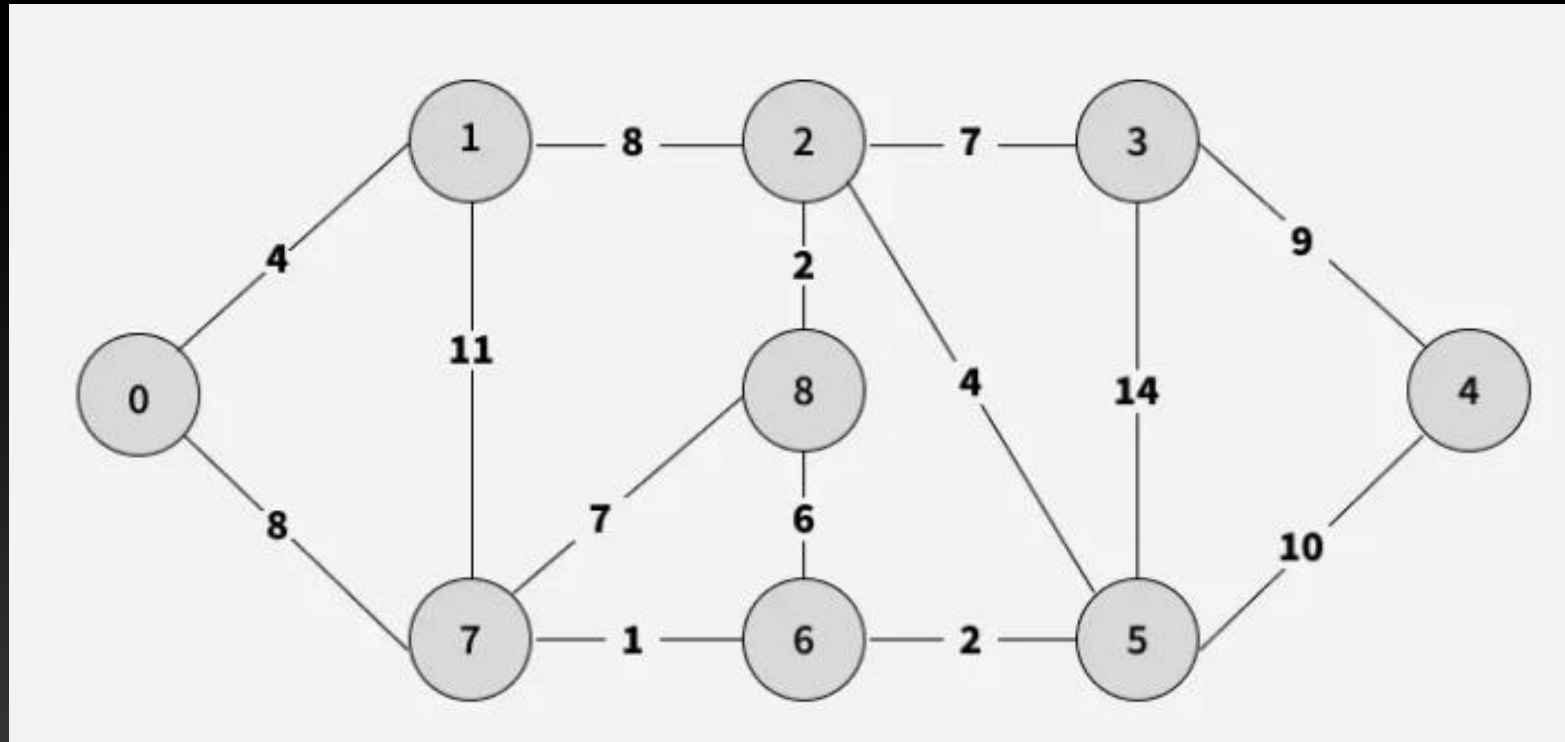
Why It Works: By the Cut Property, at each iteration the cheapest edge crossing the cut $(T, V \setminus T)$ must belong to some MST. Adding that edge cannot create a cycle, because its outside endpoint was not yet in T .

Data Structures: A min-heap (or priority queue) keyed by the current best connection weight for each outside vertex. An array $\text{key}[v]$ holding the weight of the lightest edge known so far that connects v to the tree. An array $\text{parent}[v]$ recording which tree-vertex provides that best connection.



Example of Prim's Algorithm:

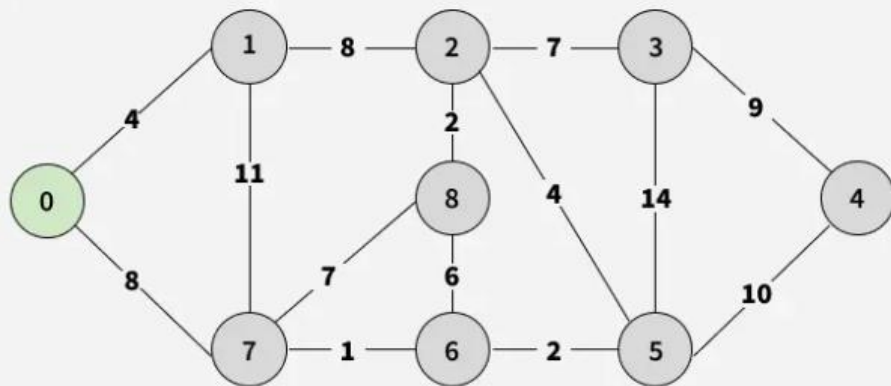
Let's examine the graph step by step with Prim's Algorithm.



The main goal is to never create a cycle.

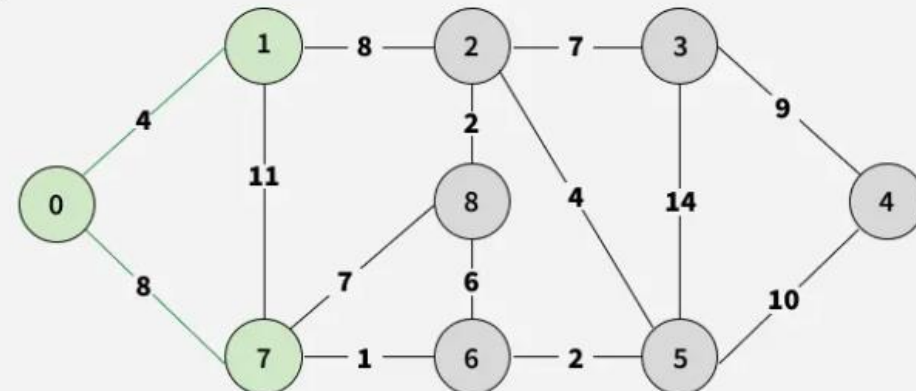
01
Step

Start with edges $\{0,1\}$ and $\{0,7\}$; pick the minimum $\{0,1\}$ and add vertex 1.



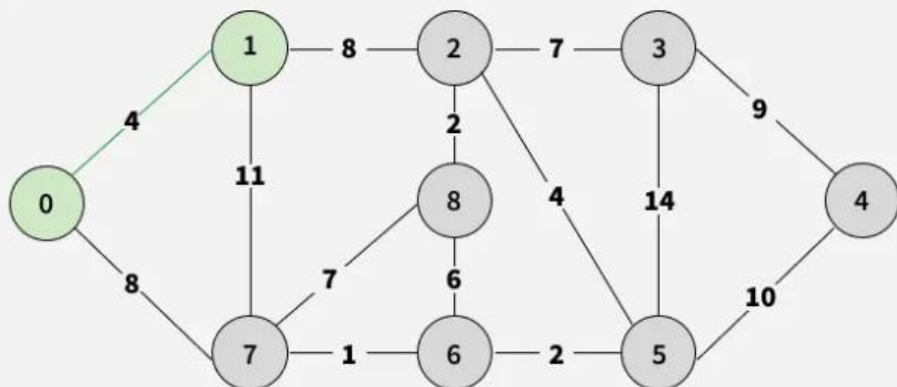
03
Step

Select $\{7,6\}$ as it has the least weight (1), adding vertex 6.



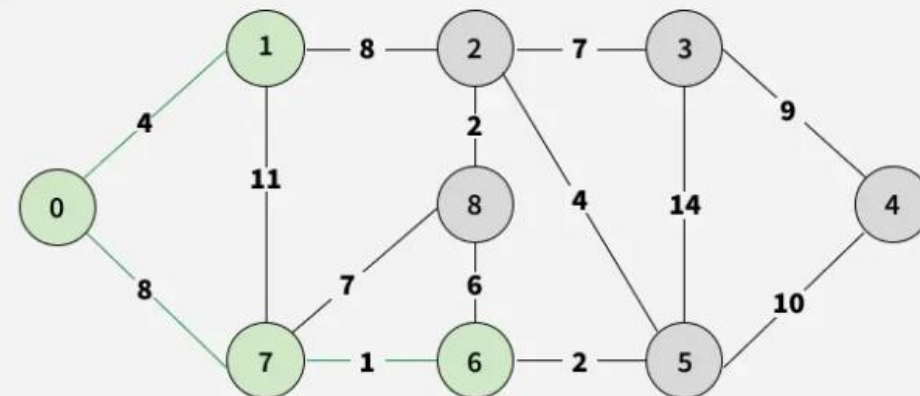
02
Step

Choose between $\{0,7\}$ and $\{1,2\}$; pick $\{0,7\}$ (or $\{1,2\}$), adding vertex 7.



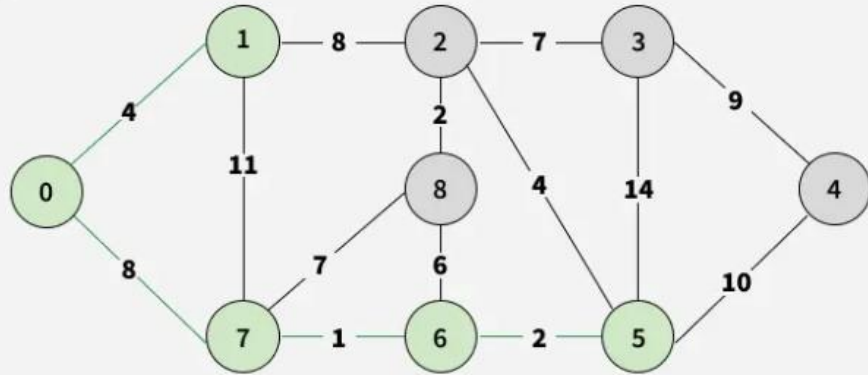
04
Step

Among $\{7,8\}$, $\{6,8\}$, and $\{6,5\}$, pick $\{6,5\}$ (weight 2) and add vertex 5.



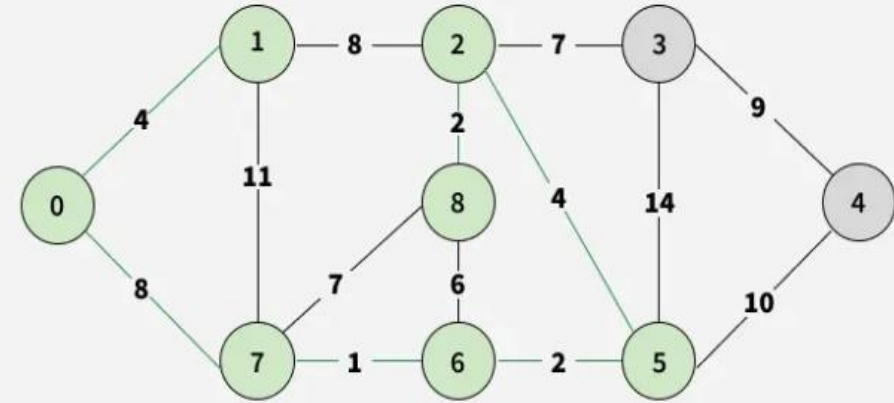
05
Step

Select {5,2} (weight 4), adding vertex 2.



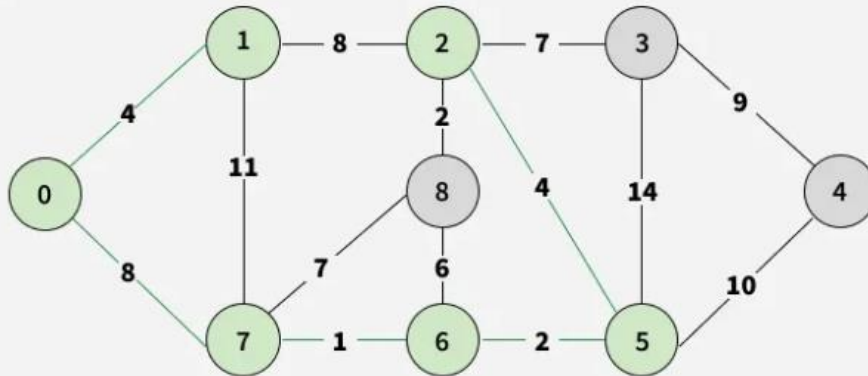
07
Step

Pick {2,3} as the smallest edge, adding vertex 3.



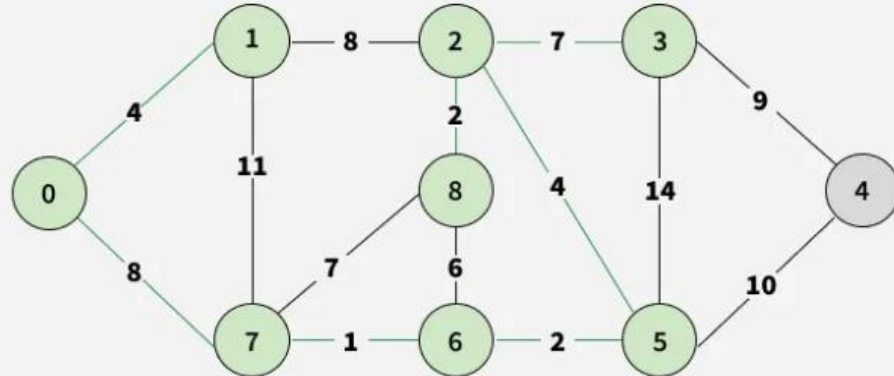
06
Step

Choose {2,8} (weight 2), adding vertex 8.



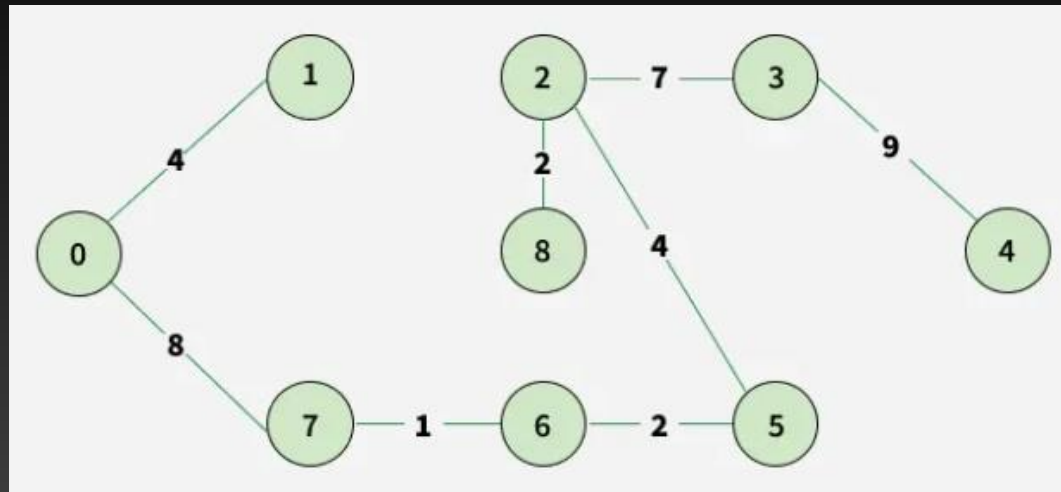
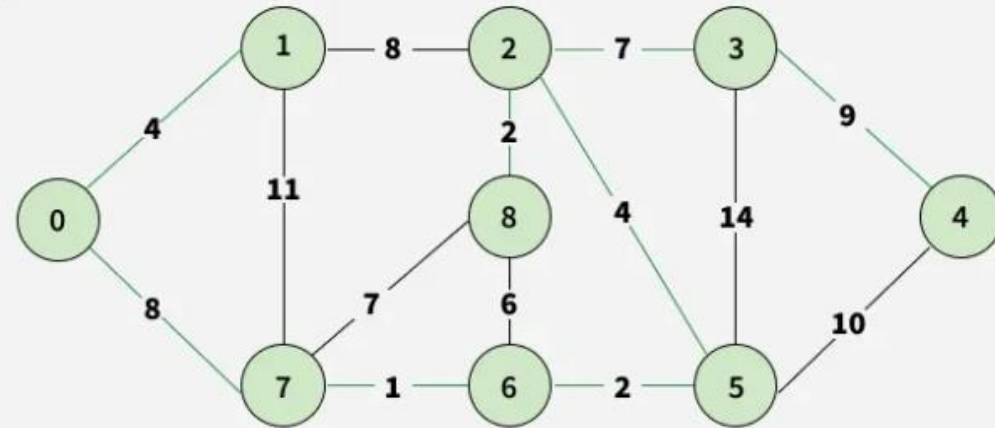
08
Step

Finally, select {3,4}, adding vertex 4.

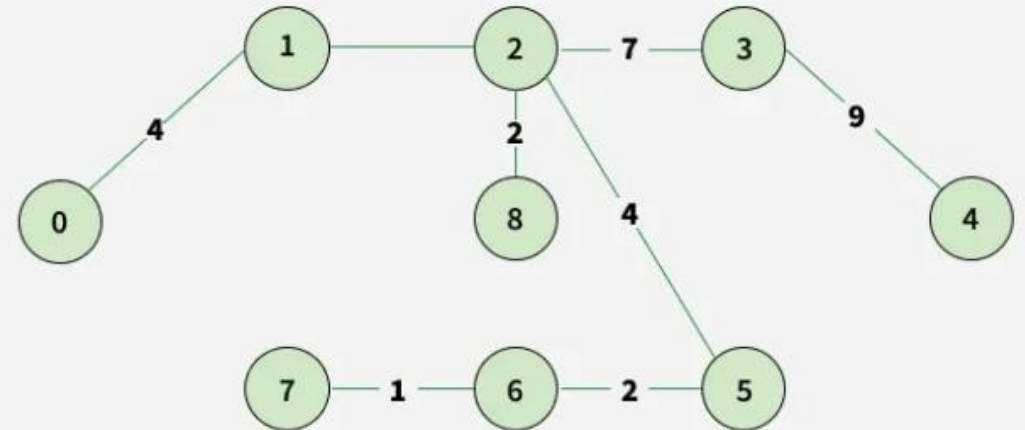


09
Step

All vertices are included, so the MST is complete.

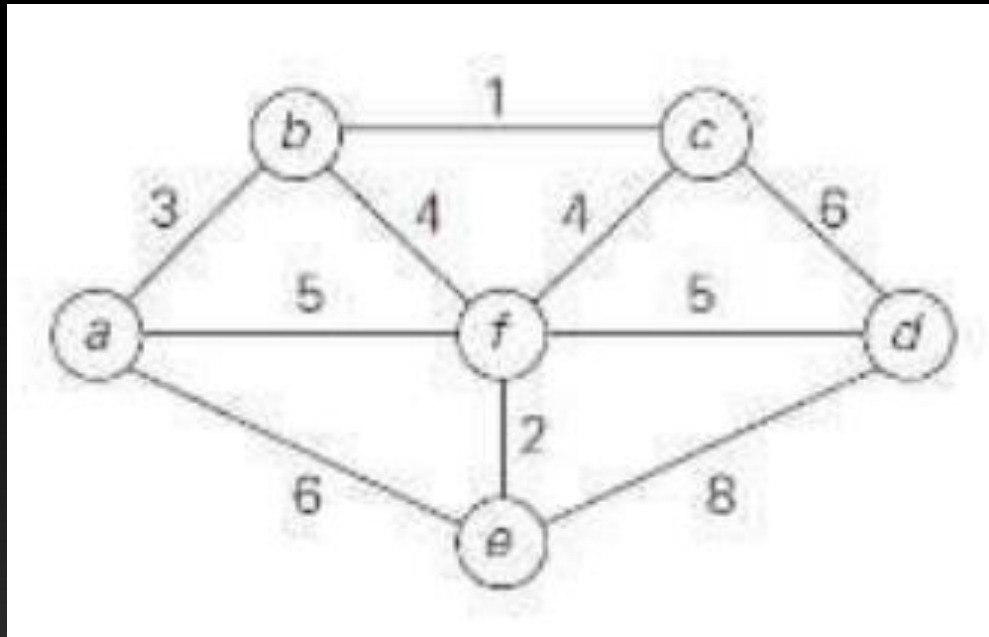


Alternative MST Structure



Example

Let's examine the graph step by step with Prim's Algorithm.



Let's create the Prim's Algorithm using Psuedo code.

ALGORITHM *Prim(G)*

//Prim's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = \langle V, E \rangle$

//Output: E_T , the set of edges composing a minimum spanning tree of G

$V_T \leftarrow \{v_0\}$ //the set of tree vertices can be initialized with any vertex

$E_T \leftarrow \emptyset$

for $i \leftarrow 1$ **to** $|V| - 1$ **do**

 find a minimum-weight edge $e^* = (v^*, u^*)$ among all the edges (v, u)
 such that v is in V_T and u is in $V - V_T$

$V_T \leftarrow V_T \cup \{u^*\}$

$E_T \leftarrow E_T \cup \{e^*\}$

return E_T

Kruskal's Algorithm:

Kruskal's algorithm is another greedy algorithm for the minimum spanning tree problem that also always yields an optimal solution. It is named Kruskal's algorithm, after Joseph Kruskal.

Kruskal's algorithm looks at a minimum spanning tree for a weighted connected graph $G = (V, E)$ as an acyclic sub graph with $|V| - 1$ edges for which the sum of the edge weights is the smallest.

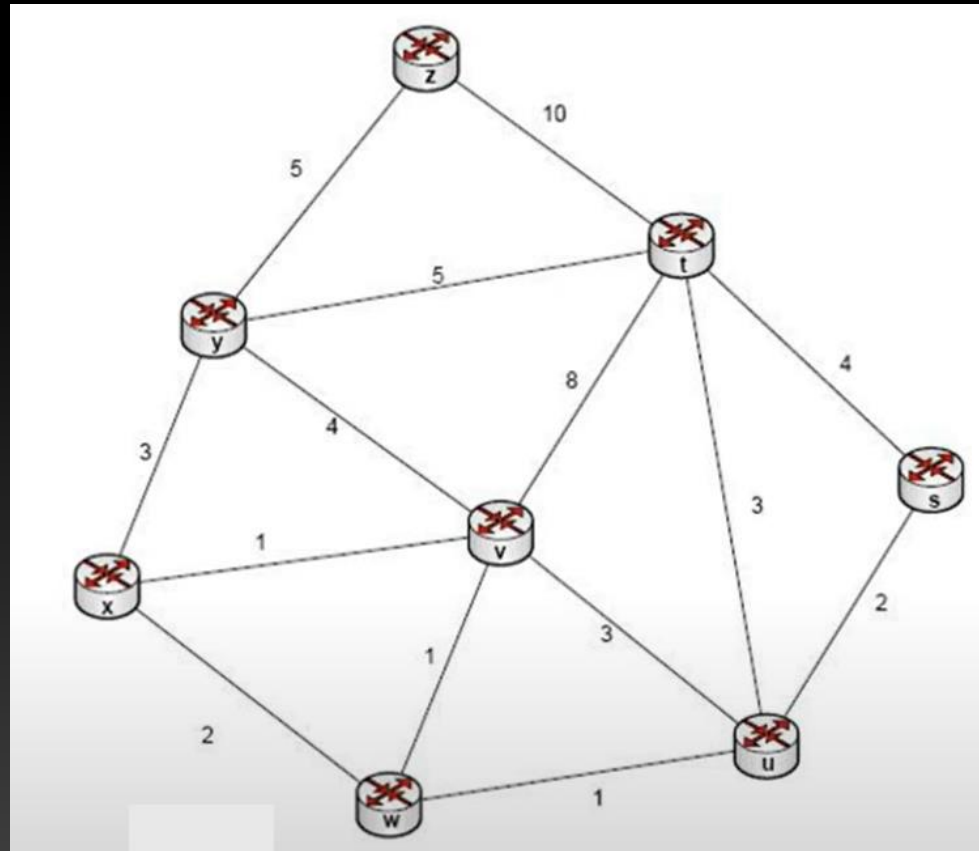
Consequently, the algorithm constructs a minimum spanning tree as an expanding sequence of sub graphs, which are always acyclic but are not necessarily connected on the intermediate stages of the algorithm.

Working: The algorithm begins by sorting the graph's edges in non-decreasing order of their weights. Then, starting with the empty subgraph, it scans this sorted list adding the next edge on the list to the current sub graph if such an inclusion does not create a cycle and simply skipping the edge otherwise.



In Shortly: There is no shorter path to every corner according to Kruksal's algorithm

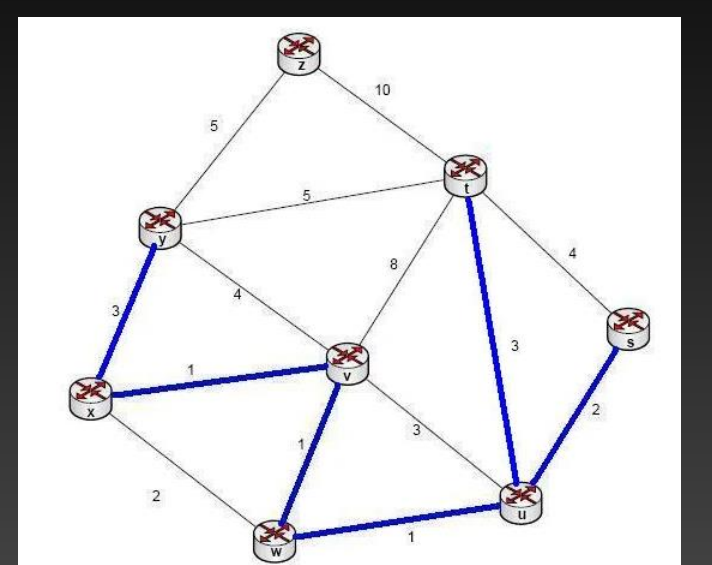
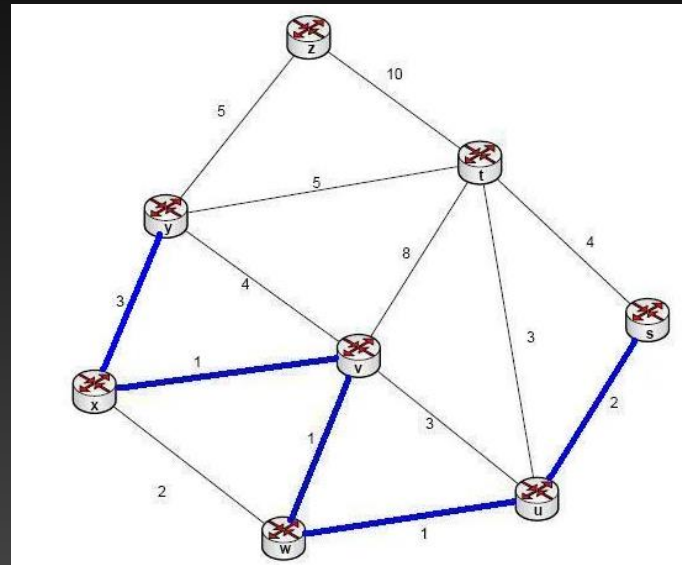
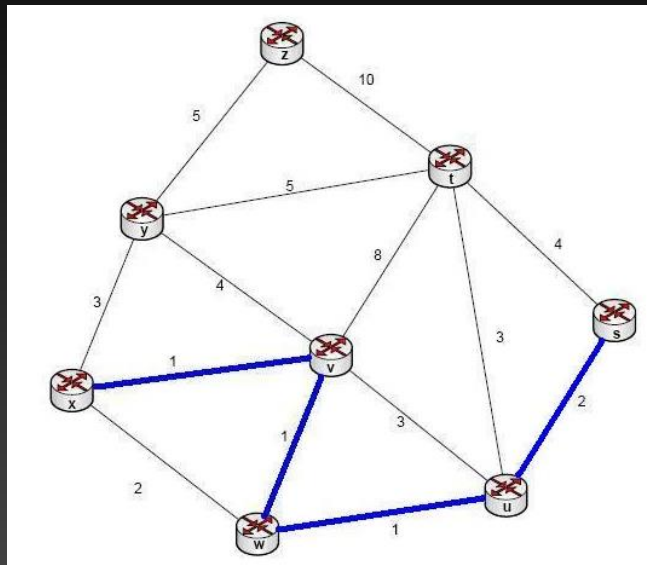
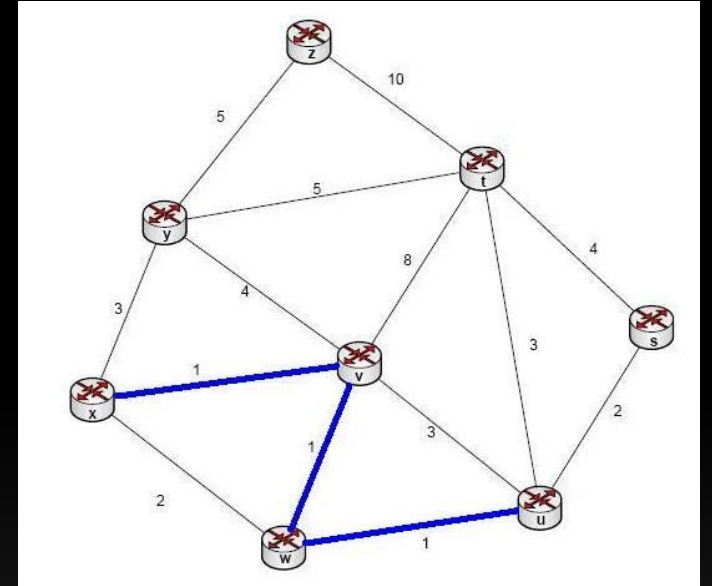
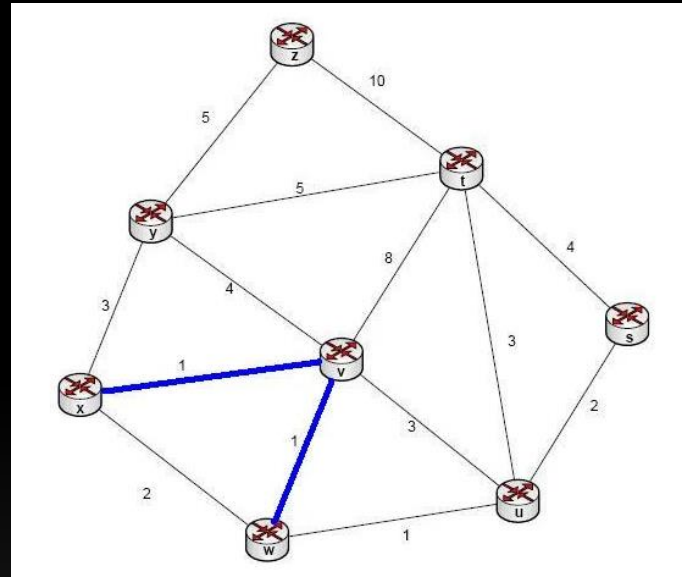
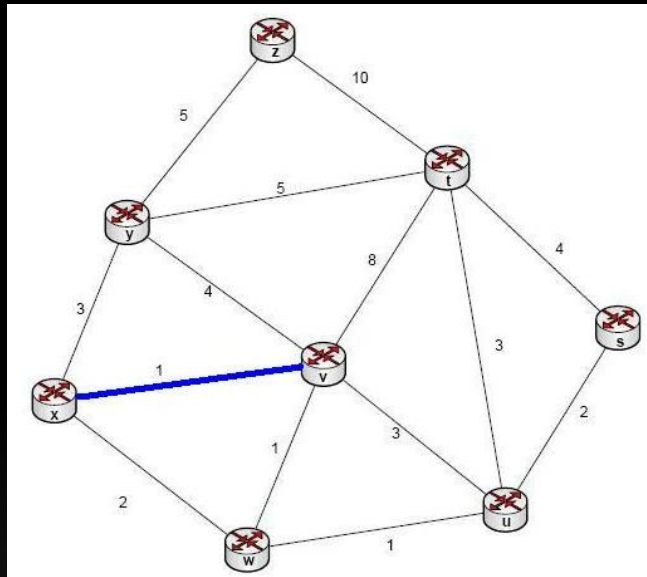
Example

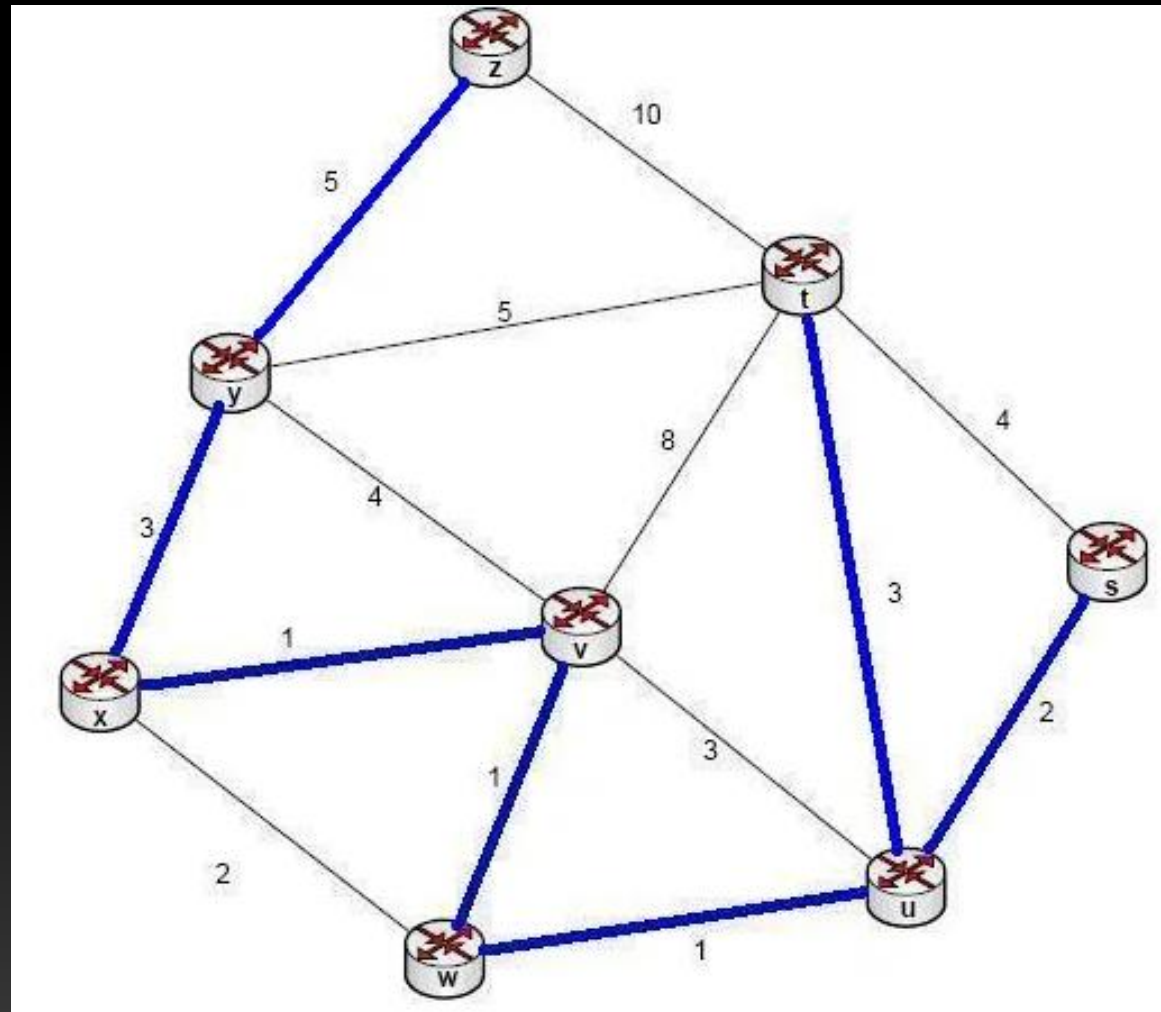


Example

In Kruskal's algorithm, all paths are listed and sorted from smallest to largest. This list is given below for the above graph:

x-v: 1
w-v: 1
w-u: 1
x-w: 2
u-s: 2
x-y: 3
t-u: 3
u-v: 3
y-v: 4
s-t: 4
y-z: 5
y-t: 5
z-t: 10





The cost of Kruksal's algorithm is 16.

Let's create the Kruskal's Algorithm using Psuedo code.

ALGORITHM *Kruskal*(G)

//Kruskal's algorithm for constructing a minimum spanning tree

//Input: A weighted connected graph $G = \langle V, E \rangle$

//Output: E_T , the set of edges composing a minimum spanning tree of G

sort E in nondecreasing order of the edge weights $w(e_{i_1}) \leq \dots \leq w(e_{i_{|E|}})$

$E_T \leftarrow \emptyset$; $ecounter \leftarrow 0$ //initialize the set of tree edges and its size

$k \leftarrow 0$ //initialize the number of processed edges

while $ecounter < |V| - 1$ **do**

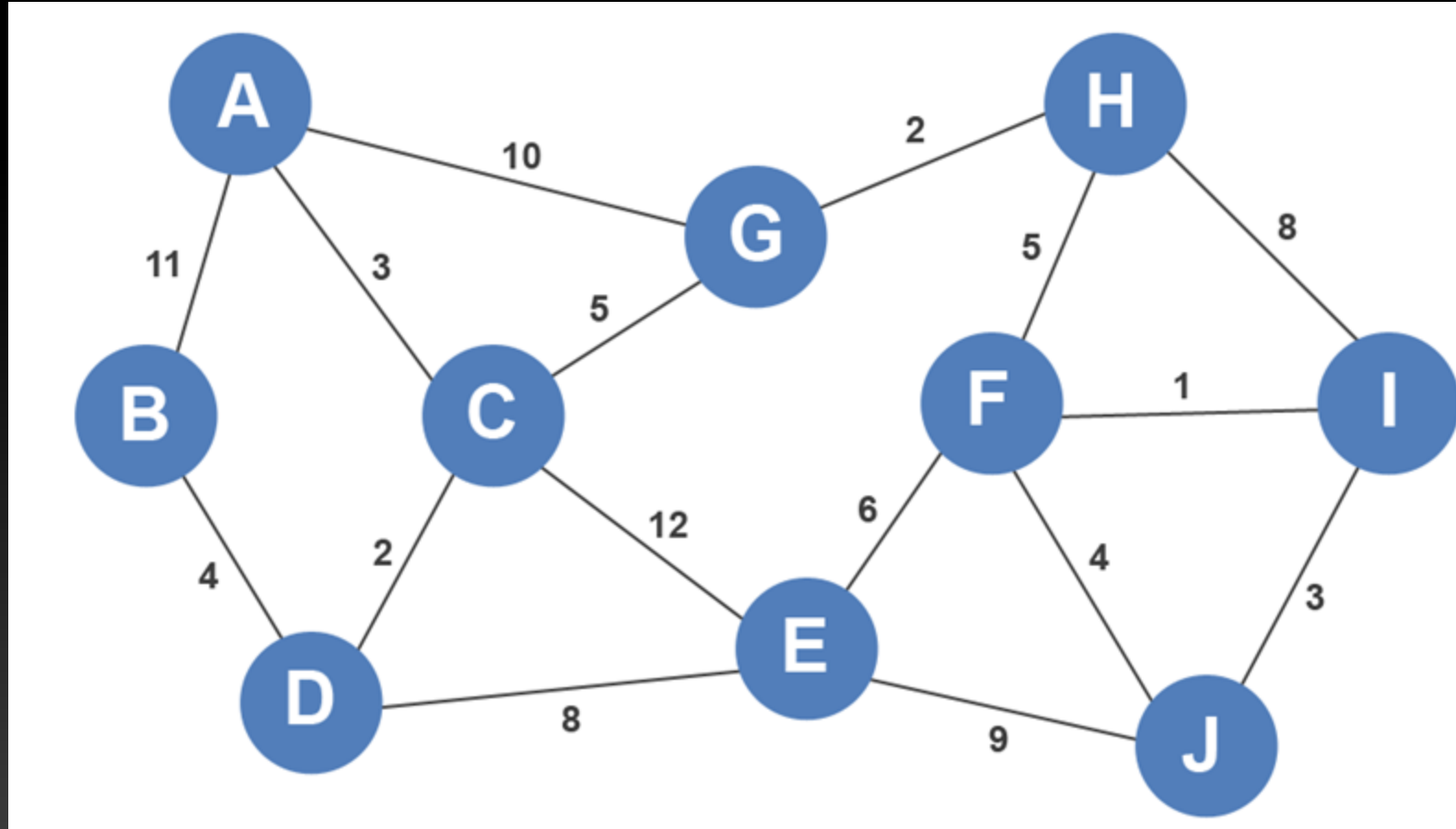
$k \leftarrow k + 1$

if $E_T \cup \{e_{i_k}\}$ is acyclic

$E_T \leftarrow E_T \cup \{e_{i_k}\}$; $ecounter \leftarrow ecounter + 1$

return E_T

Let's examine the graph step by step with Kruksal Algorithm.



Similarities and differences of Prim and Kruskal

Feature	Prim	Kruskal
Paradigm	Greedy	Greedy
Approach	Grows a single tree from a start vertex	Builds a forest, merging components
Initialization	Requires a chosen start vertex	No specific start vertex needed
Edge Selection	Chooses the smallest edge on the tree's front	Chooses the smallest edge from all remaining edges
Data Structure	Min-heap + adjacency list	Sorted edge list + Union-Find
Time Complexity	$O(E \log V)$	$O(E \log E) \approx O(E \log V)$
Preferred For	Dense graphs	Sparse graphs
Precondition	Connected, weighted graph	Connected, weighted graph
Output	Minimum spanning tree (total weight minimal)	Minimum spanning tree (total weight minimal)

Daily Life Example

Prim

Imagine food delivery service opens one restaurant first. Then it expands to nearby districts one by one with the cheapest transportation routes.
That is Prim's style: **start from one place and expand.**

Kruskal

Imagine the government wants to connect many villages with electricity lines. They check all possible connections and always choose the cheapest safe connection first.
That is Kruskal's style: **choose cheapest connections globally.**



Activity: Greedy Gridlines: Powering Istanbul from Kadıköy to Maslak

Scenario

Istanbul’s Electric Works Authority has chosen **Kadıköy Substation (S)** on the Anatolian side as the new power hub.

From this substation, they must run underground cables to seven key districts and facilities—while keeping construction costs minimal.

Your challenge is to design that network with both Kruskal’s and Prim’s algorithms, then observe how equal-cost “tie-breaks” produce different layouts (though total cost stays the same).

Label	Location
S	Kadıköy Substation
A	Beşiktaş Quarter
B	Kadıköy Residential District
C	Üsküdar District
D	Taksim Square
E	Sultanahmet (Historic Peninsula)
F	Şişli Memorial Hospital
G	Maslak Business & Industrial Zone

No.	From	To	Cost (× ₱1 000)
1	S	A	10
2	S	B	10
3	A	B	10
4	A	C	18
5	B	C	18
6	B	D	22
7	C	D	22
8	C	E	14
9	D	E	8
10	E	F	5
11	D	F	18
12	F	G	14
13	E	G	6
14	B	G	20
15	A	G	20

Student Tasks

1. Draw the Graph

Plot nodes S–G and all 15 edges with their costs on paper or using a drawing tool.

2. Run Kruskal's Algorithm

Sort edges by increasing cost.

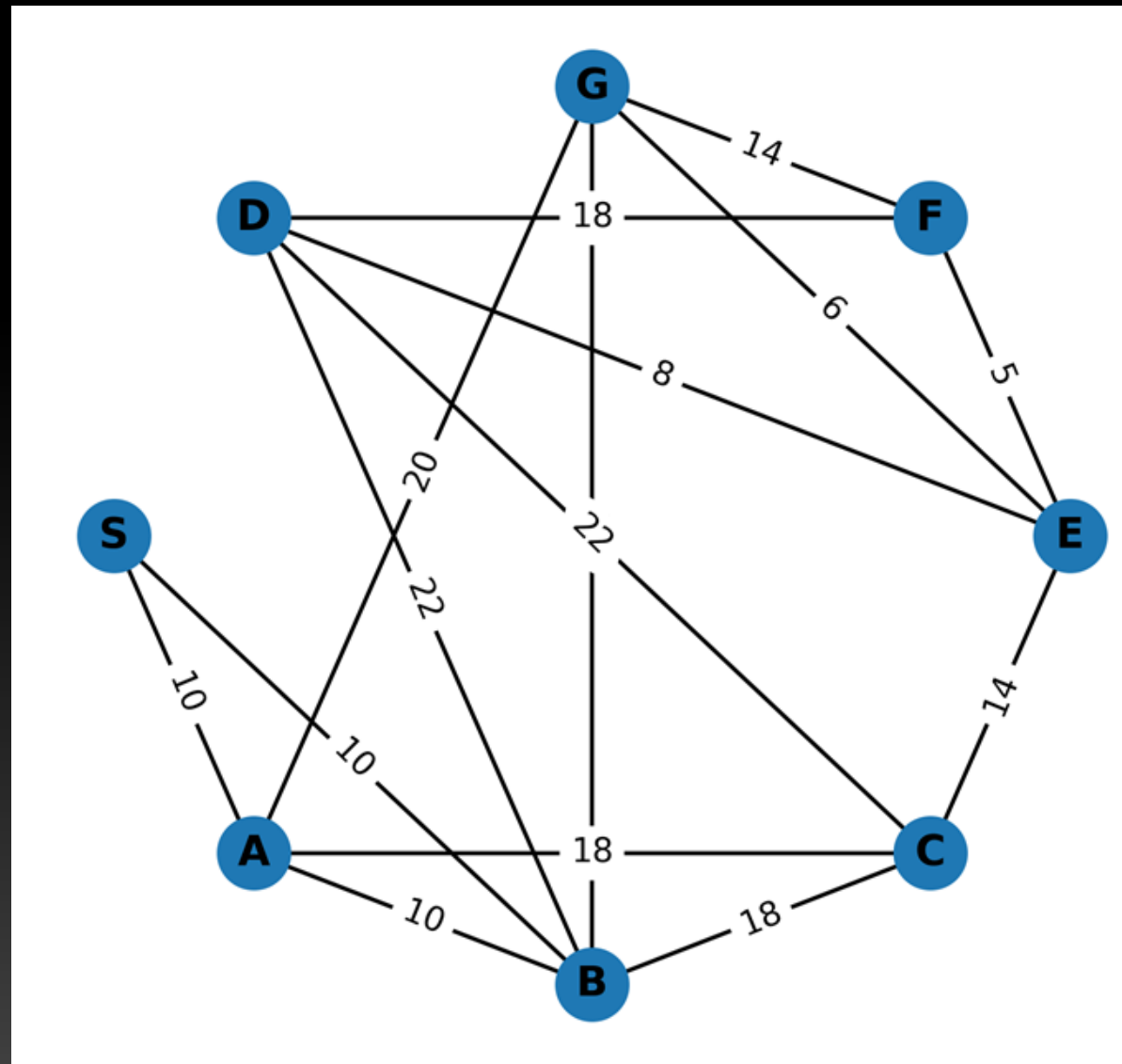
Use Union–Find to add edges (skipping those creating cycles) until you have 7 edges. Record the merge sequence, the final edge set, and total cost.

3. Run Prim's Algorithm (start at S = Kadıköy Substation)

Initialize $\text{key}[S] = 0$; all others $= \infty$. Maintain a min-heap of vertices by $\text{key}[v]$. Extract-min, update neighbors' keys/parents, repeat until all 8 nodes are in the tree. List the resulting edges and compute total cost.

4. Compare & Discuss

Show the two MST edge-sets side by side. Explain how equal-cost links (the three “10” × ₺1 000 edges, etc.) can be chosen in different order, leading to structurally different trees. Confirm both have the same total cost, and debate which layout might be more practical for Istanbul's urban constraints.



Kruskal's Algorithm

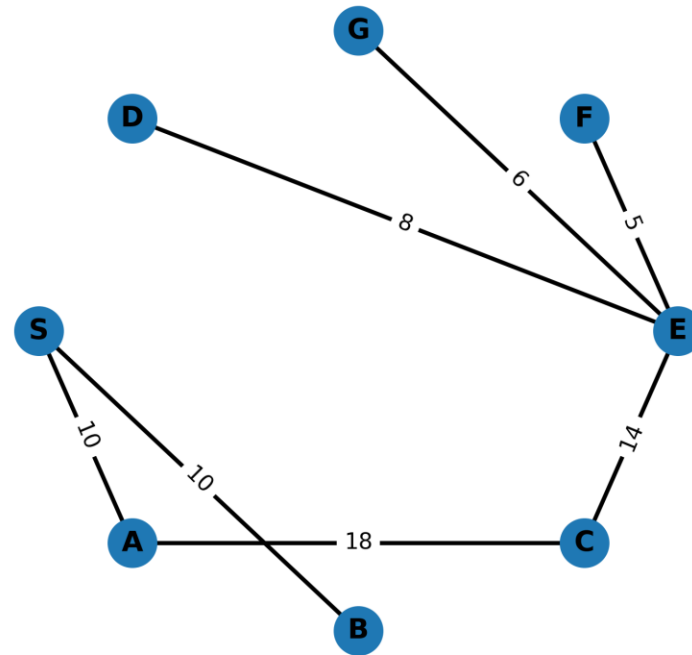
Edges Sorted by Cost

Rank	Edge	Cost (฿×1 000)
1	E–F	5
2	E–G	6
3	D–E	8
4	S–A	10
5	S–B	10
6	A–B	10
7	C–E	14
8	F–G	14
9	A–C	18
10	B–C	18
11	D–F	18
12	B–G	20
13	A–G	20
14	B–D	22
15	C–D	22

Union–Find Step-by-Step

Step	Considered Edge	Components of Endpoints	Added?	Component Merge
1	E–F (5)	{E}, {F}	Yes	{E,F}
2	E–G (6)	{E,F}, {G}	Yes	{E,F,G}
3	D–E (8)	{D}, {E,F,G}	Yes	{D,E,F,G}
4	S–A (10)	{S}, {A}	Yes	{S,A}
5	S–B (10)	{S,A}, {B}	Yes	{S,A,B}
6	A–B (10)	{S,A,B}, {S,A,B}	No	(would form a cycle)
7	C–E (14)	{C}, {D,E,F,G}	Yes	{C,D,E,F,G}
8	F–G (14)	{C,D,E,F,G}, {C,D,E,F,G}	No	(same component)
9	A–C (18)	{S,A,B}, {C,D,E,F,G}	Yes	{S,A,B,C,D,E,F,G} (all nodes linked)

Minimum Spanning Tree via Kruskal's Algorithm



After Step 9 we have 7 edges and one connected component.

Final MST Edges (Kruskal):

$$\{E-F(5), E-G(6), D-E(8), S-A(10), S-B(10), C-E(14), A-C(18)\}$$

Total Cost:

$$5 + 6 + 8 + 10 + 10 + 14 + 18 = 71 \text{ (thousand ₺)}$$

2. Prim's Algorithm (Start = S)

Initialize: $\text{key}[S]=0$, all others = ∞

$\text{parent}[v]=\text{NIL}$ for all v .

All vertices in a min-heap keyed by $\text{key}[]$.

Step	Extracted	Selected Edge	Updates to key/parent
1	S (key=0)	—	A:10←S, B:10←S
2	A (key=10)	S-A (10)	C:18←A, G:20←A
3	B (key=10)	S-B (10)	D:22←B
4	C (key=18)	A-C (18)	E:14←C
5	E (key=14)	C-E (14)	D:8←E (↓22), F:5←E, G:6←E
6	F (key=5)	E-F (5)	— (no neighbor improvements)
7	G (key=6)	E-G (6)	—
8	D (key=8)	E-D (8)	—

Now all 8 vertices are in the tree.

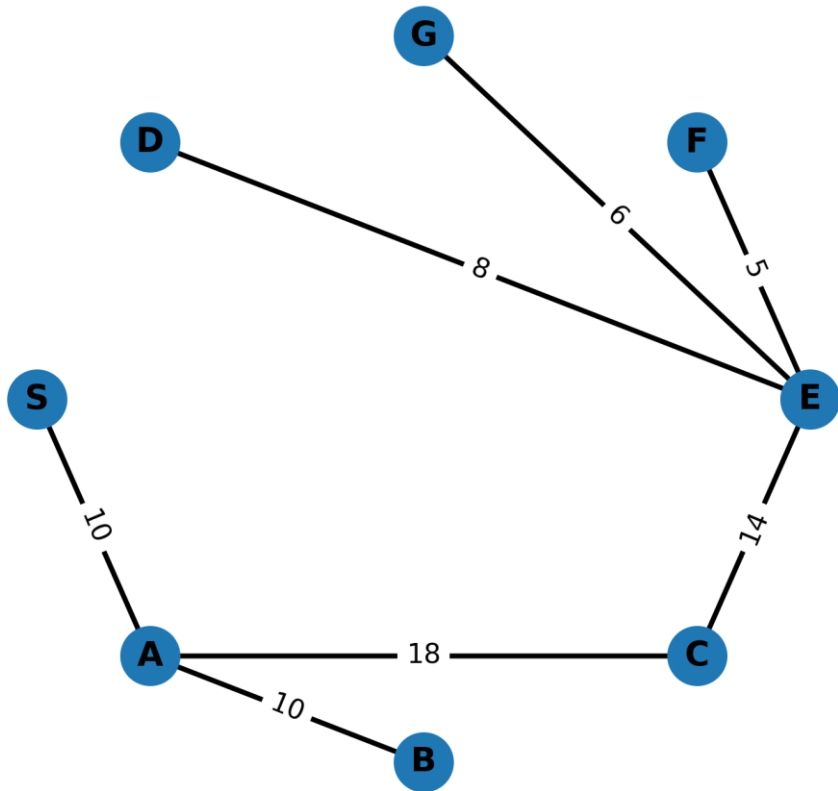
Final MST Edges (Prim):

$$\{S-A(10), S-B(10), A-C(18), C-E(14), E-F(5), E-G(6), E-D(8)\}$$

Total Cost:

$$10 + 10 + 18 + 14 + 5 + 6 + 8 = 71 \text{ (thousand ₺)}$$

Minimum Spanning Tree via Prim's Algorithm



Why the Difference?

Kruskal first examines the edge S–A, S–B, then A–B (all 10s); depending on the loop control (union-find), he can skip or choose A–B (here he can choose and construct a different path connecting S to A, B, A to C, then C to D).

Prim can start from S and first choose S–A and S–B, then directly A–C or B–C according to the min-heap order, and finally B–D or C–D.

Since the “first seen” choices they make during the tie-break (equal cost edges) are different, the resulting tree structures are also different.